

	UNINASSAU ARACAJU - CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS DISCIPLINA: TÓPICOS INTEGRADORES / LABORATÓRIO DE EMPREENDIMENTOS INOVADORES (2026.2) EVENTO: 1º MVP TÓPICOS INTEGRADORES - SILVER ECONOMY (EDITAL Nº 02/2026)
Título do Documento:	Política de Governança, Gestão de Código e Fidelidade ao Planejamento
Nome do Projeto:	BeyondTime (Plataforma de Relacionamento, Convivência e Segurança 50+)
Versão / Status:	Versão 1.0 / Documento Homologado de Governança
Instituição / Curso:	UNINASSAU Aracaju – Centro Universitário Maurício de Nassau ADS B
Líder do Projeto:	José Guilherme da Silva Nascimento
Equipe Técnica:	José Guilherme da Silva Nascimento, Abraão Silva Paixão, Paulo Rafael Cardoso Santos, Brigitte Lara Rodrigues Pereira da Silva Dantas

POLÍTICA DE GOVERNANÇA, GESTÃO DE CÓDIGO E FIDELIDADE AO PLANEJAMENTO

1. INTRODUÇÃO E ALINHAMENTO COM O EDITAL Nº 02/2026

O presente documento estabelece as diretrizes normativas de governança de software, controle de versão, gestão ágil de tarefas e garantia de fidelidade ao planejamento para o projeto **BeyondTime**. A iniciativa insere-se no âmbito do **Edital nº 02/2026 - MVP Tópicos Integradores (Silver Economy)** do Centro Universitário Maurício de Nassau (UNINASSAU Aracaju).

O objetivo primário deste regulamento interno é institucionalizar o rigor metodológico na execução do produto, assegurando que 100% do escopo planejado no Roadmap do Produto seja entregue rigorosamente dentro dos prazos, sem supressão de requisitos funcionais (RF01 ao RF14). A observância irrestrita a este documento garante o cumprimento integral dos critérios de avaliação do edital, visando a pontuação máxima (**1,0 ponto**) no quesito **"Fidelidade ao Planejamento"** na Avaliação Final.

A complexidade do ecossistema BeyondTime — composto por uma aplicação Web (React 18 / Vite), um aplicativo Mobile (React Native / Expo Framework) e um back-end corporativo distribuído em Java 17 com Spring Boot 3 — exige disciplina na gestão de artefatos de código. A governança aqui estruturada previne débitos técnicos, assegura a rastreabilidade total das entregas e blinda a equipe contra riscos de descontinuidade ou desvio de escopo durante o ciclo de desenvolvimento.

2. MODELO DE GESTÃO DE CÓDIGO (GITFLOW SIMPLIFICADO)

2.1 Estrutura e Hierarquia de Branches

O repositório de código-fonte centralizado no GitHub adota o modelo de versionamento **GitFlow Simplificado**, projetado para garantir isolamento de funcionalidades e estabilidade nas entregas de produção. A topologia das ramificações (*branches*) obedece à seguinte hierarquia estrita:

- **Branch `main`**: Ramificação de produção contendo exclusivamente código estável, testado e homologado correspondente às Releases oficiais do edital (Release 1, Release 2 e Release 3). É expressamente proibido qualquer commit direto nesta branch.
- **Branch `develop`**: Ramificação de integração contínua. Serve como tronco principal para a consolidação do trabalho diário da equipe. Todo código pré-homologado deve ser mesclado primeiro em `develop` antes de avançar para a `main`.
- **Branches de Feature (`feature/rf-xx`)**: Ramificações temporárias destinadas ao desenvolvimento individual de requisitos. É obrigatório nomear a branch relacionando-a diretamente ao código do Requisito Funcional (exemplo: `feature/rf-01-autenticacao`, `feature/rf-06-ux-defensiva`, `feature/rf-10-assistente-ia`).

2.2 Política de Pull Requests e Peer Review

A mesclagem de código de uma branch `feature` para a branch `develop` exige obrigatoriamente a abertura de um **Pull Request (PR)** no GitHub. O processo de integração deve seguir os seguintes critérios:

- **Aprovação Obrigatória por Peer Review**: Nenhum Pull Request será integrado sem a revisão formal de código e aprovação explícita de pelo menos 1 (um) outro membro da equipe de desenvolvimento diferente do autor da solução.
- **Verificação de Compilação**: O código proposto deve compilar sem erros e passar em todos os testes unitários integrados antes da aprovação do PR.
- **Execução de Merge Sem Fast-Forward**: As mesclagens devem manter o histórico de commits limpo e auditável, garantindo a rastreabilidade da origem de cada funcionalidade.

2.3 Convenção de Mensagens de Commit (Conventional Commits)

Para manter o histórico do repositório padronizado e auditável pela banca examinadora, todas as mensagens de commit devem seguir estritamente o padrão internacional **Conventional Commits** na estrutura `<tipo>(<escopo>): <descrição em tom imperativo>`. Os tipos permitidos são:

- `feat`: Adição de novo requisito ou funcionalidade de negócio (ex: `feat(rf-05): implementa algoritmo de match por afinidade`).
- `fix`: Correção de defeito ou bug no código existente (ex: `fix(rf-01): corrige expiração incorreta de token JWT`).
- `docs`: Alteração exclusiva em documentações acadêmicas ou técnicas (ex: `docs(roadmap): atualiza matriz de rastreabilidade ABNT`).
- `style`: Ajuste visual de layout ou formatação de código sem alteração na regra de negócio (ex: `style(rf-09): ajusta tamanho do texto para alto contraste`).

3. GESTÃO ÁGIL DE TAREFAS E FLUXO KANBAN

3.1 Mapeamento das Raias do Quadro Kanban (GitHub Projects)

O acompanhamento operacional das atividades utiliza o método Kanban gerenciado através do **GitHub Projects**. O fluxo de trabalho é dividido em 4 raias estritas, com limite de trabalho em progresso (*Work In Progress - WIP*) para evitar gargalos e sobrecarga:

1. **A Fazer (To Do):** Contém o backlog de tarefas da Onda atual aguardando alocação.
2. **Em Desenvolvimento (In Progress):** Tarefas sob codificação ativa. Aplica-se a regra de **Limite de WIP de no máximo 2 tarefas simultâneas por desenvolvedor**.
3. **Em Revisão / QA (Code Review):** Tarefas com Pull Request aberto aguardando aprovação de Peer Review e testes de aceitação.
4. **Concluído (Done):** Código integrado com sucesso na branch `deve1op` e aprovado nos critérios de aceite do requisito.

3.2 Rastreabilidade dos Requisitos Funcionais (RF01 a RF14)

Cada cartão (*Issue*) criado no quadro Kanban deve possuir vinculação direta e bidirecional com a Matriz de Rastreabilidade do Roadmap do Produto (RF01 ao RF14). É obrigatório registrar no título e na descrição da Issue o código do requisito correspondente, o responsável pela execução, a Onda de entrega e os critérios de aceite definidos no projeto.

4. PROTOCOLO DE DESVIO ZERO E TRATATIVA DE MUDANÇAS DE ESCOPO

4.1 Regra de Inviolabilidade de Requisitos

Fica estabelecida a **Regra de Inviolabilidade de Requisitos**: nenhum Requisito Funcional (RF01 a RF14) constante na documentação aprovada do projeto poderá ser descartado, ignorado ou suprimido do escopo do MVP. A descontinuidade não autorizada de qualquer funcionalidade acarretará a perda imediata da pontuação no critério de Fidelidade ao Planejamento.

4.2 Fluxo de Gestão de Mudanças Técnicas

Na ocorrência de imprevistos tecnológicos críticos (tais como descontinuidade de APIs de terceiros, limitações de orçamento de infraestrutura ou incompatibilidades de frameworks), a equipe deverá acionar obrigatoriamente o seguinte protocolo de mitigação em até 24 horas:

1. **Convocação de Reunião Extraordinária:** O Líder de Projeto reúne a equipe para analisar o impacto técnico e mapear alternativas viáveis.
2. **Readequação de Complexidade Preservando a Funcionalidade:** A equipe reajustará a arquitetura de implementação para uma solução de menor complexidade técnica, garantindo que a entrega funcional do requisito para o usuário 50+ permaneça 100% preservada (exemplo: substituir uma API paga por uma biblioteca Open Source local).
3. **Emissão e Submissão da Ata Técnica:** Qualquer readequação técnica deve ser registrada em uma *Ata de Registro de Mudança Técnica* e submetida formalmente à aprovação do docente/orientador da disciplina antes do encerramento da Sprint.

5. MATRIZ SINTÉTICA DE REGRAS DE GOVERNANÇA

A Tabela 1 sintetiza o conjunto de regras normativas de governança, relacionando os pilares operacionais, os responsáveis diretos e os critérios de aceite auditáveis.

Tabela 1 – Matriz Sintética de Diretrizes de Governança e Controle de Qualidade.

Pilar da Governança	Regra / Diretriz Operacional	Mecanismo de Controle / Ferramenta	Responsável / Critério de Aceite
Modelo de Versionamento	Uso estrito de GitFlow Simplificado (main , develop , feature/rf-xx).	Proteção de branches no GitHub com bloqueio de commit direto na main / develop .	Toda a Equipe Técnica. Zero commits diretos nas branches protegidas.
Qualidade e Review	Obrigatória abertura de Pull Request com aprovação de pelo menos 1 revisor.	Peer Review no GitHub com aprovação de revisão técnica antes do merge.	Desenvolvedores e Revisor. PR aprovado sem conflitos de integração.
Padronização de Commits	Mensagens em formato Conventional Commits (feat , fix , docs , style , refactor).	Auditoria visual do histórico do repositório no GitHub e hooks de commit.	Toda a Equipe Técnica. 100% dos commits no padrão normativo.
Gestão Ágil (Kanban)	Acompanhamento em 4 raias com limite de WIP de no máximo 2 tarefas por dev.	Quadro visual interativo gerenciado através do GitHub Projects.	Líder de Projeto. Sem violação de limite de WIP nas raias.
Rastreabilidade	Vinculação direta e obrigatória entre cada Issue no Kanban e os RFs (RF01 a RF14).	Inclusão da tag [RF-XX] nas Issues, PRs e commits do projeto.	Líder e Desenvolvedores. 100% dos requisitos mapeados e auditáveis.
Fidelidade ao Planejamento	Protocolo de Desvio Zero: proibição absoluta de supressão de qualquer requisito funcional.	Emissão de Ata Técnica formal submetida ao docente em caso de ajuste tecnológico.	Líder do Projeto. Entrega dos 14 RFs na Avaliação Final (1,0 pt).

Fonte: Elaborado pelos autores (2026).

RESUMO DAS FERRAMENTAS DE GOVERNANÇA DO BEYONDTIME

- Gestão de Repositório & Versionamento: GitHub (GitFlow, Branch Protection Rules).
- Gerenciamento Ágil & Kanban: GitHub Projects (Rastreabilidade RF01-RF14, Limite WIP).
- Padrão de Qualidade & Revisão: Conventional Commits, Peer Review Obrigatório.
- Garantia de Escopo: Protocolo de Desvio Zero de Requisitos, Ata de Registro Técnico.

6. DECLARAÇÃO DE COMPROMISSO E FIDELIDADE AO PLANEJAMENTO

Nós, membros da equipe de desenvolvimento do projeto **BeyondTime**, declaramos para os devidos fins acadêmicos e institucionais nosso compromisso integral com o cumprimento desta Política de Governança, Gestão de Código e Fidelidade ao Planejamento.

Reafirmamos o compromisso de entregar com excelência técnica todos os 14 Requisitos Funcionais (RF01 ao RF14) previstos no Roadmap do Produto, respeitando as regras de versionamento no Git, o fluxo Kanban de tarefas, a qualidade do código e as normas do Edital nº 02/2026 - MVP Tópicos Integradores (Silver Economy) da UNINASSAU Aracaju, visando a obtenção da nota máxima no critério de avaliação de Fidelidade ao Planejamento (1,0 ponto).